

IterClean: An Iterative Data Cleaning Framework with Large Language Models

Wei Ni^{†‡1}, Kaihang Zhang^{†2}, Xiaoye Miao^{†#3}, Xiangyu Zhao^{‡4}, Yangyang Wu^{*5}, Jianwei Yin^{*6}

[†]Center for Data Science, Zhejiang University, Hangzhou, China

[#]The State Key Lab of Brain-Machine Intelligence, Zhejiang University, Hangzhou, China

[‡]School of Data Science, City University of Hong Kong, Hong Kong, China

^{*}Software College, Zhejiang University, Ningbo, China

^{*}College of Computer Science, Zhejiang University, Hangzhou, China

{¹wei.ni, ²zjuzkh, ³miaoxy, ⁵zjuwuyy}@zju.edu.cn ⁴xianzhao@cityu.edu.hk ⁶zjuyjw@cs.zju.edu.cn

ABSTRACT

In the era of generative artificial intelligence, the accuracy of data is paramount. Erroneous data often leads to faulty outcomes and economic detriments. Previous cleaning methods employ a *sequential* detect-repair paradigm, leaving over half of the errors unsolved in real scenarios. We introduce IterClean, an *iterative* data cleaning framework leveraging large language models (LLMs). Utilizing an iterative mechanism, the framework employs a two-step process: *data labeling* and *iterative data cleaning*. With few labeled data, IterClean leverages an iterative cleaning process involving an *error detector*, an *error verifier*, and an *error repairer* to significantly enhance the cleaning performance. Extensive experiments across four datasets demonstrate that, IterClean achieves an F1 score that is up to three times higher than the best state-of-the-art approaches requiring only 5 labeled tuples.

CCS CONCEPTS

• Database → Data Cleaning.

KEYWORDS

Data cleaning, error detection, error repair, large language models

ACM Reference Format:

Wei Ni^{†‡1}, Kaihang Zhang^{†2}, Xiaoye Miao^{†#3}, Xiangyu Zhao^{‡4}, Yangyang Wu^{*5}, Jianwei Yin^{*6}. 2024. IterClean: An Iterative Data Cleaning Framework with Large Language Models. In *ACM Turing Award Celebration Conference 2024 (ACM-TURC '24)*, July 05–07, 2024, Changsha, China. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3674399.3674436>

1 INTRODUCTION

In today's world, applications across the globe are deeply reliant on data, particularly in the burgeoning field of generative artificial intelligence (GAI) [6, 9, 33]. However, various errors appear within the data, such as *rule violations* (data that break predefined rules), *pattern violations* (data in unexpected patterns), and *outliers* (data that significantly deviate from the others). Such errors can gravely

undermine the efficacy of data analysis, leading to flawed analytical outcomes and potentially immense economic losses [16, 28, 29, 44].

Data cleaning, as an important means to ensure the data quality, is usually conducted in two *sequential* two steps: *error detection* and *error repair* [9, 25, 26, 32]. The errors are first detected, and then repaired into the right ones within detection results. In this paradigm, errors within a *single* error type can be effectively mitigated using *hard* constraints, including pre-defined rules and patterns. However, errors comprising *multiple* error types, cannot be effectively addressed with these methods due to two reasons. First, designing a unified hard restriction for all error types is challenging. Hence, the cleaning algorithms often rely on machine learning (ML) models without hard restrictions, leading to their potential failure in addressing *all* errors. Previous research shows that, state-of-the-art repair methods leave an average of 48% of errors unresolved in real-world scenarios [25, 36]. Second, it is observed that, current repair tools tend to introduce more errors than they correct [32], suggesting a need for careful re-evaluation of repair decisions.

Moreover, existing cleaning methods mainly rely on pre-defined rules or labeled data [9, 25]. This not only leads to high costs, but also limits their cleaning ability. The latent right values may not exist in the original data or rules, making these algorithms fail to conduct a correct repair. While some strategies incorporate knowledge bases and master data [11, 15], they generally lack the flexibility needed to adapt to data from diverse domains. Furthermore, the process of labeling data or writing rules demands an intensive understanding of the data, which typically necessitates expert knowledge or a large amount of human effort.

To cope with limitations present in existing data cleaning tools, in this paper, we propose a *novel iterative data cleaning framework* that leverages the capabilities of *large language models* (LLM), termed *IterClean*. It consists of *three* roles based on LLM, namely error detector, verifier, and repairer. The error detector identifies errors in the data and explains the corresponding reasons. To alleviate the hallucinations of LLMs, within the detection results and corresponding explanations, the verifier validates the errors by cross-referencing the detector's explanations with the data context. The error repairer infers the latent right values for the erroneous data based on the detection results. The repaired data are re-examined by the detector. This process continues until the detector finds no errors or reaches the defined rounds.

To enhance the understanding of the relationship between tuples and ensure consistent results, we develop a *batch prompting* strategy based on the data grouping, according to the subject attribute [39],

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

ACM-TURC '24, July 05–07, 2024, Changsha, China

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1011-7/24/07

<https://doi.org/10.1145/3674399.3674436>

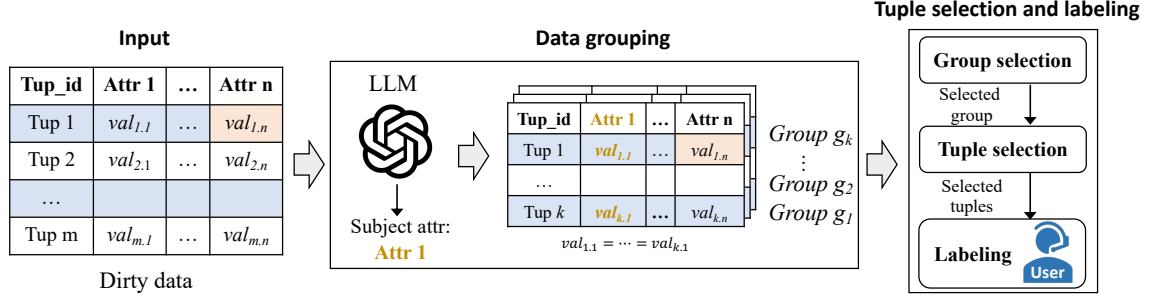


Figure 1: The process of data selection for labeling in IterClean. The cell colored in light orange has a wrong value.

which is the attribute containing the main entity names. Moreover, to reduce the human cost and maximize the utilization of few labeled data, we present a *two-stage tuple selection* strategy for user labeling. In the first stage, we select the batch with the greatest level of dispersion. Then, inspired by [21], the outlier points in the batch are selected to label based on Isolation Forest [23].

The main contributions of this paper are as below.

- We introduce a novel *iterative* data cleaning framework, which allows for bidirectional iterative interactions between the error detection and the error repair processes, making use of LLM.
- We develop a two-stage tuple selection strategy for user labeling, and a batch prompting strategy based on the subject column, effectively cleaning the data with only 5 labeled tuples.
- Extensive experiments within both real-world and synthetic datasets demonstrate the superior performance of IterClean, compared with the state-of-the-art methods.

The rest of the paper is structured as follows. Section 2 provides related works. The problem statement is introduced in Section 3. Section 4 details the proposed framework. Experimental evaluations are presented in Section 5. Finally, Section 6 concludes the paper.

2 RELATED WORK

Data Cleaning. Data cleaning typically involves two steps: error detection and error repair. Error detection has two study lines, i.e., *rule-driven* and *data-driven* error detection. The rule-based error detection algorithms identify error values based on given rules like FDs [2, 4], DCs [10, 13], conditional functional dependencies [14], and manually defined parameters [3, 36]. In contrast, data-driven error detection algorithms identify errors based on (weak) labeled data, without relying on predefined rules [17, 21, 26, 35]. Error repair algorithms usually employ internal or external information. Internally, they utilize integrity constraints [37] and the data distribution [25, 27, 36, 45]. Externally, they incorporate information from master data [15], knowledge base [11], and downstream data analysis models [21]. Existing cleaning methods mainly operate these two steps sequentially, thus overlooking the introduced errors in the repair process.

Large language models. Recent studies have indicated that enhancing the scale of pre-trained language models (PLMs), such as increasing the model or data size, frequently enhances their capacity to perform effectively on a variety of downstream tasks. Unlike PLMs, large language models (LLMs) have demonstrated emergent abilities [24, 41, 46], the capacity for in-context learning [12, 30], adherence to instructions [34, 38, 40], and incremental reasoning [42]. These capabilities show that LLMs can accurately

adapt to the task based on just a few illustrative examples. Presently, the primary emphasis in LLM-based data cleaning involves creating cleaning prompts for LLMs [31], incorporating LLMs into data curation workflows [7], and fine-tuning LLM for a variety of table-related tasks [22]. They do not consider the detection-repair steps in the data cleaning task.

3 PROBLEM STATEMENT

In this paper, we study the data cleaning problem in relational datasets. Similar as the definition in previous literature [25, 36], given an instance I of a relation R characterized by a set of attributes $Attrs = \{A_1, A_2, \dots, A_n\}$. Each $t \in I$ comprises a set of cells represented by $Cells[t] = A_i[t]$, corresponding to a distinct attribute in $Attrs$. For each cell c , its unknown true value is denoted by v_c^* , its initial observed value is v_c , and its estimated true value is $\hat{v}_c$. The data cleaning algorithm aims to make the estimated $\hat{v}_c$ equal to v_c^* for all cells c .

4 METHODOLOGY

In this section, we first provide an overview of our proposed data cleaning framework IterClean. Then, we explain the data selection strategy for labeling, and the iterative cleaning process in detail.

4.1 Solution Overview

There are two primary steps in the workflow of IterClean, namely data labeling and iterative data cleaning, as illustrated in Figure 1 and Figure 2, respectively. Initially, the data labeling process is conducted. Subsequently, with these labeled data, the original dirty data is cleaned through an iterative cleaning process.

Firstly, IterClean aims to label few tuples, so that the LLM can understand the unique distribution of errors within the dataset, and capture the relation between attributes. To comprehensively display the attribute relations, considering each tuple actually describes an entity, we initially group data according to the *subject attribute* [39], which is the attribute containing the main entity names. Then, to make the selected tuples possibly be erroneous, IterClean employs a *strategic* approach that integrates data grouping with a two-stage selection strategy to enhance outcomes.

For the iterative data cleaning process, utilizing the labeled tuples and data groups, IterClean conducts batch prompting. Each data group is assigned a specific prompt, which is subsequently processed by the error detector to identify and explain errors. The verifier then reviews the results considering the provided data and explanations. Next, the repairer infers the corrected data for errors. This iterative process repeats until no errors are detected or a predefined number of iterations is reached.

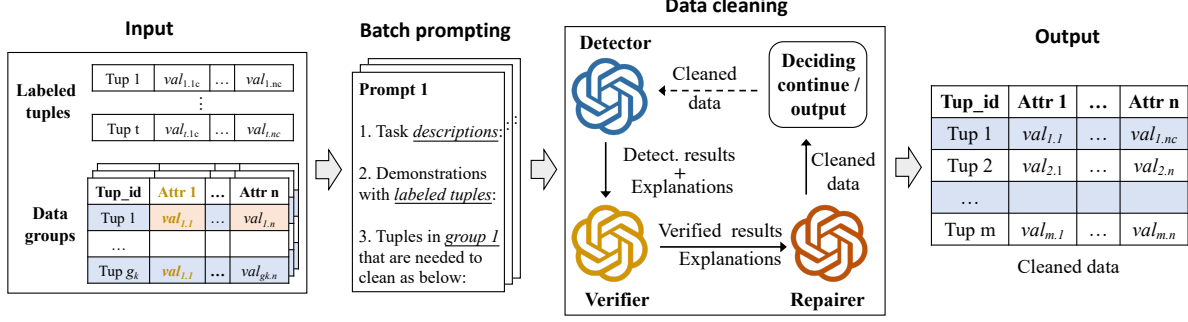


Figure 2: The iterative data cleaning process of IterClean.

4.2 Data Selection for Labeling

As depicted in Figure 1, in the data selection process, the *subject attribute* is first identified by the LLM. The dirty data is grouped by identical values in the subject attribute. Data features are then extracted, followed by a two-stage selection for error labeling.

4.2.1 Data Grouping. The process of grouping data starts with the initial input of dirty data into the LLM, specifically querying the subject attribute. After getting the results, the dirty data is divided into groups, where each group contains tuples with identical values in the subject attribute.

4.2.2 Feature Representation. To make the selected data representative for the errors, inspired by previous studies [19, 26], for each cell value, we consider three types of frequency, namely *value frequency*, *vicinity frequency*, and *pattern frequency*. These frequencies correspond to the types of errors commonly encountered, such as outliers (w.r.t. value frequency), rule violations (w.r.t. vicinity frequency), and deviations from patterns (w.r.t. pattern frequency).

For a cell value $A_i[t]$ of tuple t , value and vicinity frequency are defined as below:

$$f_{val}(A_i[t]) = \frac{\text{count}(A_i[t])}{|A_i|}$$

$$f_{vi}(A_i[t]) = \frac{\{\text{count}(A_i[t]|A_j[t]) \mid A_j \neq A_i, \forall A_j \in \text{Attrs}\}}{|A_j[t]|}$$

where $\text{count}(A_i[t])$ denotes the occurrence number of value $A_i[t]$. $\text{count}(A_i[t]|A_j[t])$ represents how often the value $A_j[t]$ determines $A_i[t]$. $|A_i|$ and $|A_j[t]|$ refer to the A_i cardinality and the occurrence number of $A_j[t]$.

For the pattern frequency, inspired by previous research for pattern violation detection [19], we consider *three* pattern generalization levels of the value $A_i[t]$, for the generalized pattern pat_v , its frequency is defined as:

$$f_{pat_v}(A_i[t]) = \frac{\text{count}(pat_v(A_i[t]))}{|pat_v(A_i)|}$$

So far, for each tuple, IterClean concatenates the above features of all the cell values, presenting as follows:

$$\text{Feat}(t) = \{[f_{val}(A_i[t]), f_{vi}(A_i[t]), f_{pat_1}(A_i[t]), f_{pat_2}(A_i[t]), f_{pat_3}(A_i[t])] \mid \forall A_i \in \text{Attrs}\}$$

4.2.3 Two-Stage Selection Strategy. The above features suggest the value character within the whole data. Thus, the outlier features are more likely to contain errors. Inspired by this, during data

selection, we first select the group exhibiting high variability, as greater variability implies inconsistency and a higher probability of errors. With the selected data group, we then specifically select the possible outlier tuples.

Group selection. To evaluate the homogeneity of the data group, the concept of within-cluster sum of squares (WCSS) is employed [20]. WCSS effectively measures the variability or compactness of the data group and represents the sum of the squared distances between each data point and the group centroid. Given that, the WCSS for a particular group g_t consisting of multiple tuples can be formalized as follows:

$$wcss(g_t) = \sum_{t_i \in g_t} \|\text{Feat}(t_i) - \mu\|^2 \quad (1)$$

$$\mu = \frac{1}{|g_t|} \sum_{t_i \in g_t} \text{Feat}(t_i) \quad (2)$$

where $\text{Feat}(t_i)$ is the feature for the i -th tuple in g_t , and μ is the centroid of all tuple features in g_t . Then, we can select the data group within the highest wcss. The selected index k of all groups $\mathbb{G}$ can be calculated by:

$$k = \arg \max_{g_k \in \mathbb{G}} wcss(g_k) \quad (3)$$

Tuple selection. Following the group selection, to select the outlier tuples in the group, we employ the prevailing Isolation Forest model [23], which detects anomalies using binary trees in linear time complexity.

4.2.4 Labeling Process. For each tuple selected, the user examines the value within every attribute to ascertain its correctness. If any errors are found, the user specifies the correct values to replace the erroneous ones.

4.3 Iterative Data Cleaning

Figure 2 illustrates the labeling process, to improve the scalability and results' consistency, IterClean adopts a batch prompting approach according to the data group [8]. This approach enables LLMs to understand data relations and clean data groups in parallel.

4.3.1 Batch Prompting. For each data group, we formalize the prompt involving three parts, task descriptions, demonstrations, and the specific data group.

Task descriptions. The task description includes a delineation of the assigned role and a detailed content of input along with the task explanations. It also specifies the output format. The task

description T can be represented as:

$$T = \{r, e, f\}$$

where r represents the defined role within the task, e corresponds to the explanations of the input and the task to be conducted, and f outlines the required format for the output.

Demonstrations. Subsequently, the provision of correctly labeled tuples as demonstrations significantly facilitates the cleansing process. These examples serve as an instructional guide, thereby augmenting the efficacy of the methodological approach. Mathematically, the demonstration D can be defined as a set of tuples:

$$D = \{(v_1, y_1), (v_2, y_2), \dots, (v_l, y_l)\}$$

where (v_i, y_i) pairs an attribute value v_i with its correct label y_i .

Data group. Lastly, the data group G_i for cleaning is illustrated. This specification is crucial for understanding the relation of the data set subject to the cleaning process. The prompt for each data group G_i can be formatted as:

$$Prompt = \{T, D, G_i\}$$

4.3.2 Iterative Process. In the data cleaning process, three pivotal roles emerge, namely error detector, error verifier, and error repairer, aiming at the detection, validation, and correction of errors.

Error Detector. The error detector operates at the forefront of the data cleaning process, detecting errors within the data group. Detailed explanations for each identified error are provided, establishing a foundational basis for subsequent stages. The error detector D can be represented as:

$$D(Prompt) = \{err, r_{err}\}$$

where $Prompt$ is generated from the batch prompting process, err is the identified errors, and r_{err} is the rationale for these errors.

Error Verifier. Subsequently, to alleviate the *hallucination* of LLM, the error verifier validates the errors detected by the Detector, ensuring that only errors with valid rationales are acknowledged for correction. This entails a comprehensive analysis of the data related to the detected error values, alongside the rationales provided. The verification function can be symbolized as:

$$V(Prompt, D(Prompt)) = \{err_i \mid err_i \in err, r_{err_i} \text{ is valid.}\}$$

Essentially, the verifier filters the errors err_i in detection results err based on the validity of the rationale r_{err_i} provided by the detector.

Error Repairer. After verification, the repairer is tasked with the correction of validated errors, a process that may encompass imputing missing values and amending inconsistent entries. The repair function can be represented as:

$$R(Prompt, V(Prompt, D(Prompt))) = \{cleaned_{err}\}$$

The cleaned data set $\{cleaned_{err}\}$ is then fed into the error detector, further finding the *wrongly cleaned* data and previously *unfound* errors. This iterative cycle of detection, verification, and repair continues until the detector can no longer find errors, or a predefined number of iterations is achieved. Ultimately, the output is the cleaned data.

Table 1: The used datasets in the experiments.

Name	#Tuples	#Attr	Error rate	Error type
Hospital	1,000	20	3%	O, PV, RV
Flights	2,376	7	30%	O, PV, RV
Beers	2,410	11	16%	O, PV, RV
Billionaire	2,615	22	10%	O, PV, RV

5 EXPERIMENTS

We conduct a series of experiments to evaluate IterClean in comparison with the state-of-the-art data cleaning methods across various datasets and error rates. We also explore the impact of the data selection and batch prompting strategies.

Datasets. In the experiments, we adopt four datasets, i.e., Hospital, Flights, Beers, and Billionaire, as depicted in Table 1. *Hospital* and *Flights* are well-known benchmarks in data cleaning, with a high degree of data redundancy. *Beers* [18] is a real-world dataset cleaned by the owner. We also crafted a dirty *Billionaire* [43] dataset by injecting various errors randomly into the original data with the publicly available code from BigDaMa [5]. Following previous works [25, 32], we introduce more common real-world errors such as typos, missing values, Gaussian noise, and rule violations. These datasets are chosen for their variety of common error types, including outliers, pattern violations, and rule violations.

Baselines. We evaluate IterClean with the following competing data cleaning methods. For data repair, **Baran** [25] is a data repair system integrating various corrector models with few labeled data. **HoloClean** [36] is a holistic data repair framework driven by probabilistic inference. For error detection, we incorporate **Raha** [26], recognized for its leading error detection performance. For baseline methods, we initiate the process with Raha to identify errors, which are then repaired by repair algorithms within Raha's results. Once repaired, the data undergoes a subsequent Raha analysis for additional error detection. This cyclical process persists until reaching the predetermined number of iterations.

Evaluation Metrics. We assess the effectiveness of data cleaning approaches using established metrics like precision, recall, and the F1-score (F1), all of which have been previously employed in the literature [1, 36]. Precision is the percentage of errors correctly fixed out of those addressed. Recall measures the proportion of errors fixed out of all actual errors. The F1-score is the harmonic mean of precision and recall.

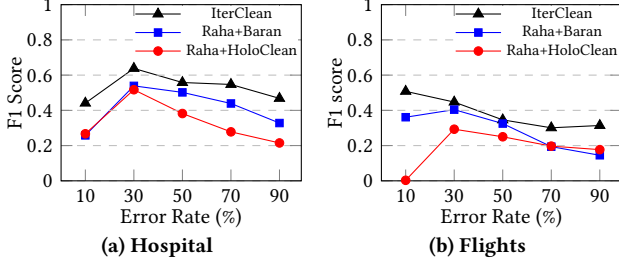
In our studies, *GPT-3.5-turbo* is utilized for IterClean. We have set a maximum of 3 iteration rounds for all methods. Throughout our assessment phases, we consistently label 5 tuples. All experimental procedures are carried out on a server outfitted with an Intel(R) Xeon(R) Gold 6326 CPU operating at 2.90GHz, complemented by 512GB RAM, and functioning under Ubuntu 20.04.6 LTS.

5.1 Effectiveness of the Iterative Mechanism

We evaluated IterClean's performance against baseline methods on various datasets, as shown in Table 2. IterClean surpassed most methods in F1 scores after just one data cleaning round. While Raha and Baran only slightly outperformed IterClean on the Beers dataset, IterClean's strong performance elsewhere indicates its effectiveness in data cleaning without iterations.

Table 2: Framework effectiveness compared with the baselines.

Methods	Hospital			Flights			Beers			Billionaire		
	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1
Raha + HoloClean (round 1)	1.000	0.059	0.111	0.348	0.369	0.358	0.000	0.000	0.000	0.007	0.013	0.009
Raha + Baran (round 1)	0.924	0.120	0.212	0.711	0.313	0.440	0.607	0.483	0.538	0.328	0.034	0.062
IterClean (round 1)	0.697	0.580	0.633	0.763	0.317	0.448	0.729	0.344	0.468	0.578	0.096	0.165
Raha + HoloClean (round 3)	1.000	0.061	0.115	0.354	0.379	0.366	0.000	0.000	0.000	0.018	0.033	0.024
Raha + Baran (round 3)	0.924	0.120	0.212	0.741	0.313	0.440	0.607	0.483	0.538	0.328	0.034	0.062
IterClean (round 3)	0.585	0.845	0.691	0.719	0.508	0.595	0.686	0.537	0.603	0.593	0.185	0.281

**Figure 3: Methods effectiveness v.s. various error rates.**

Following three rounds of the iterative data cleaning process, Raha paired with HoloClean shows slight improvement. However, the combination of Raha and Baran demonstrates almost no enhancement. This can be primarily attributed to their reliance on the labeled tuples, which remain unchanged across the cleaning iterations and are correctly repaired in the first round. As a result, there are no errors to label in the subsequent rounds, preventing Baran’s correction models from learning new knowledge. IterClean, on the other hand, showed marked improvements, particularly in the Hospital and Billionaire datasets, where its F1 score more than *doubled* compared to the other two approaches, driven by significant recall increases. This indicates its effectiveness and superiority in correcting more errors over successive iterations. Despite a slight precision-recall trade-off, IterClean’s iterative use of LLMs efficiently cleans data with multi-type errors, underscoring its value for reliable data-driven decision-making. HoloClean’s perfect precision in the Hospital dataset is possibly due to the limited cleaned cells, which also leads to a low recall performance.

5.2 The Effect of Error Rates

To examine the error rate influence on method performance, on the *benchmark* dataset Hospital and Flights, we generate several dirty datasets with incremental error rates derived from the clean version based on the previous error generation method mentioned in the datasets’ introduction.

Figure 3 displays the performance of each approach at different error rates ranging from 10% to 90%. It can be observed that IterClean consistently achieves the highest F1 scores across all error rate scenarios. It also shows greater robustness, with performance not experiencing a significant decline as the error rate increases. An anomalously low point is observed where all systems appear to perform inadequately at a 10% error rate. Besides, a general trend of enhanced cleaning efficiency is evident as the error rate progresses from 10% to 30%. These results may be due to the relatively small number of selected tuples used. In scenarios with lower error rates, insufficient labeled tuples might prevent the error detection and correction models from learning error distributions.

Table 3: Ablation study for the proposed strategies.

Methods	Hospital			Flights		
	Precision	Recall	F1	Precision	Recall	F1
IterClean	0.585	0.845	0.691	0.719	0.508	0.595
w/ Rand. Select.	0.585	0.652	0.617	0.517	0.330	0.403
w/ Rand. B-P.	0.413	0.633	0.544	0.202	0.137	0.164
w/ Rand. Select. & B-P.	0.451	0.686	0.500	0.182	0.122	0.146

5.3 Impact of Labeling and Batch Prompting

To validate the effectiveness of our Labeling and Batch Prompting strategies, we conducted ablation studies on the Hospital and Flights benchmark datasets. For comparison, we replaced our proposed data selection and batch prompting strategies with random ones, referred to as ‘Rand. Select.’ and ‘Rand. B-P.’ respectively.

The ablation study highlighted in Table 3 confirms the importance of IterClean’s strategic data selection and grouping for its superior performance. In the Hospital dataset, IterClean achieves an F1-score of 0.691, significantly outperforming the variations with random selection, random grouping, and both random strategies. Similarly, for the Flights dataset, IterClean achieves an F1-score of 0.595, compared to 0.403, 0.164, and 0.146 for the random variations, respectively. The findings particularly point out that the batch prompting strategy has a greater impact than data selection one. This underlines IterClean’s enhancements in data cleaning efficacy compared to random strategies.

6 CONCLUSION

In this paper, we present IterClean, an innovative iterative data cleaning framework utilizing LLMs. IterClean excels in detecting and repairing data errors through its iterative paradigm involving error detection, verification, and repair process. Extensive experiments with both real-world and synthetic datasets show that IterClean outperforms the existing methods, achieving superior recall and F1 scores. This framework highlights the potential of LLMs and the iterative mechanism in enhancing data quality.

ACKNOWLEDGMENTS

This work is partly supported by Fundamental Research Funds for the Central Universities (226-2024-00030), Beijing Life Science Academy (2024900CB0080, 2023000CB0020), and Henan Institute of Chinese Engineering Development Strategies (2023HENZDB01). Xiangyu Zhao is partially supported by Research Impact Fund (R1015-23), APRC-CityU New Research Initiatives (9610565), CityU-HKIDS Early Career Research Grant (9360163), Huawei (Huawei Innovation Research Program), Tencent (CCF-Tencent Open Fund, Tencent Rhino-Bird Focused Research Program), Ant Group (CCF-Ant Research Fund, Ant Group Research Fund), CCF-BaiChuan-Ebtech Foundation Model Fund, and Kuaishou. Xiaoye Miao is the corresponding author.

REFERENCES

- [1] Mohamed Abdelaal, Christian Hammacher, and Harald Schöning. 2023. REIN: A comprehensive benchmark framework for data cleaning methods in ML pipelines. In *EDBT*. 499–511.
- [2] Serge Abiteboul, Richard Hull, and Victor Vianu. 1995. *Foundations of Databases: The Logical Level*. Addison-Wesley Longman Publishing Cooperation, Inc.
- [3] Laure Berti-Equille, Tamraparni Dasu, and Divesh Srivastava. 2011. Discovery of complex glitch patterns: A novel approach to quantitative data cleaning. In *ICDE*. 733–744.
- [4] George Beskales, Ihab F. Ilyas, and Lukasz Golab. 2010. Sampling the repairs of functional dependency violations under hard constraints. *Proceedings of the VLDB Endowment* 3, 1–2 (2010), 197–207.
- [5] BigDaMa. 2023. Error generator. <https://github.com/BigDaMa/error-generator>.
- [6] Ali Borji. 2023. Generated faces in the wild: Quantitative comparison of stable diffusion, midjourney and DALL-E 2. *ArXiv Preprint ArXiv:2210.00586* (2023).
- [7] Zui Chen, Lei Cao, Sam Madden, Tim Kraska, Zeyuan Shang, Ju Fan, Nan Tang, Zihui Gu, Chunwei Liu, and Michael Cafarella. 2024. SEED: Domain-specific data curation with large language models.
- [8] Zhoujun Cheng, Jungo Kasai, and Tao Yu. 2023. Batch Prompting: Efficient Inference with Large Language Model APIs. *ArXiv Preprint ArXiv:2301.08721* (2023). [arXiv:2301.08721](https://arxiv.org/abs/2301.08721)
- [9] Xu Chu, Ihab F. Ilyas, Sanjay Krishnan, and Jiannan Wang. 2016. Data cleaning: Overview and emerging challenges. In *SIGMOD*. 2201–2206.
- [10] Xu Chu, I. F. Ilyas, and P. Papotti. 2013. Holistic data cleaning: Putting violations into context. In *ICDE*. 458–469.
- [11] Xu Chu, John Morcos, Ihab F. Ilyas, Mourad Ouzzani, Paolo Papotti, Nan Tang, and Yin Ye. 2015. KATARA: A data cleaning system powered by knowledge bases and crowdsourcing. In *SIGMOD*. 1247–1261.
- [12] Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Zhiyong Wu, Baobao Chang, Xu Sun, Jingjing Xu, Lei Li, and Zhifang Sui. 2023. A survey on in-context learning. *ArXiv Preprint ArXiv:2301.00234* (2023).
- [13] Amr Ebaid, Ahmed Elmagarmid, Ihab F. Ilyas, Mourad Ouzzani, Jorge-Arnulfo Quijane-Ruiz, Nan Tang, and Si Yin. 2013. NADEEF: A generalized data cleaning system. *Proceedings of the VLDB Endowment* 6, 12 (2013), 1218–1221.
- [14] Wenfei Fan, Floris Geerts, Xibei Jia, and Anastasios Kementsietsidis. 2008. Conditional functional dependencies for capturing data inconsistencies. *ACM Transactions on Database Systems* 33, 2, Article 6 (2008), 48 pages.
- [15] Wenfei Fan, Jianzhong Li, Shuai Ma, Nan Tang, and Wenyan Yu. 2012. Towards certain fixes with editing rules and master data. *The VLDB Journal* 21, 2 (2012), 213–238.
- [16] Shuang Hao, Nan Tang, Guoliang Li, Jian He, Na Ta, and Jianhua Feng. 2017. A novel cost-based model for data repairing. *IEEE Transactions on Knowledge and Data Engineering* 29, 4 (2017), 727–742.
- [17] Alireza Heidari, Joshua McGrath, Ihab F. Ilyas, and Theodoros Rekatsinas. 2019. HoloDetect: Few-shot learning for error detection. In *SIGMOD*. 829–846.
- [18] J.N. Hould. [n.d.]. Craft beers dataset. <https://www.kaggle.com/nickhould/craft-cans>. Accessed: 2024-05-18.
- [19] Zhipeng Huang and Yeye He. 2018. Auto-detect: Data-driven error detection in tables. In *SIGMOD*. 1377–1392.
- [20] Paul Kay and Terry Regier. 2003. Resolving the question of color naming universals. *Proceedings of the National Academy of Sciences* 100, 15 (2003), 9085–9089.
- [21] Sanjay Krishnan, Michael J. Franklin, Ken Goldberg, and Eugene Wu. 2017. Boost-Clean: Automated error detection and repair for machine learning. *ArXiv Preprint ArXiv:1711.01299* (2017).
- [22] Peng Li, Yeye He, Dror Yashar, Weiwei Cui, Song Ge, Haidong Zhang, et al. 2024. Table-GPT: Table fine-tuned GPT for diverse table tasks. *Proceedings of the ACM on Management of Data* 2, 3, Article 176 (2024), 28 pages.
- [23] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. 2012. Isolation-based anomaly detection. *ACM Transactions on Knowledge Discovery from Data* 6, 1 (2012), 39 pages.
- [24] Qidong Liu, Xian Wu, Xiangyu Zhao, Yuanshao Zhu, Derong Xu, Feng Tian, and Yefeng Zheng. 2023. Moelora: An moe-based parameter efficient fine-tuning method for multi-task medical applications. *ArXiv Preprint ArXiv:2310.18339* (2023).
- [25] Mohammad Mahdavi and Ziawasch Abedjan. 2020. Baran: Effective error correction via a unified context representation and transfer learning. *Proceedings of the VLDB Endowment* 13, 12 (2020), 1948–1961.
- [26] Mohammad Mahdavi, Ziawasch Abedjan, Raul Castro Fernandez, Samuel Madden, Mourad Ouzzani, Michael Stonebraker, and Nan Tang. 2019. Raha: A configuration-free error detection system. In *SIGMOD*. 865–882.
- [27] Xiaoye Miao, Yangyang Wu, Lu Chen, Yunjun Gao, Jun Wang, and Jianwei Yin. 2021. Efficient and effective data imputation with influence functions. *Proceedings of the VLDB Endowment* 15, 3 (2021), 624–632.
- [28] Xiaoye Miao, Yangyang Wu, Lu Chen, Yunjun Gao, and Jianwei Yin. 2023. An experimental survey of missing data imputation algorithms. *IEEE Transactions on Knowledge and Data Engineering* 35, 7 (2023), 6630–6650.
- [29] Xiaoye Miao, Yangyang Wu, Jun Wang, Yunjun Gao, Xudong Mao, and Jianwei Yin. 2021. Generative semi-supervised learning for multivariate time series imputation. In *AAAI*. 8983–8991.
- [30] Sewon Min, Mike Lewis, Luke Zettlemoyer, and Hannaneh Hajishirzi. 2022. MetaCL: Learning to learn in context. *ArXiv Preprint ArXiv:2110.15943* (2022).
- [31] Avanika Narayan, Ines Chami, Laurel Orr, and Christopher Ré. 2022. Can foundation models wrangle your data? *Proceedings of the VLDB Endowment* 16, 4 (2022), 738–746.
- [32] Wei Ni, Xiaoye Miao, Xiangyu Zhao, Yangyang Wu, Shuwei Liang, and Jianwei Yin. 2024. Automatic Data Repair: Are We Ready to Deploy? *Proceedings of the VLDB Endowment* (2024).
- [33] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, et al. 2022. Training language models to follow instructions with human feedback. *ArXiv Preprint ArXiv:2203.02155* (2022).
- [34] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, et al. 2022. Training language models to follow instructions with human feedback. *ArXiv Preprint ArXiv:2203.02155* (2022). [arXiv:2203.02155](https://arxiv.org/abs/2203.02155)
- [35] Minh Pham, Craig A. Knoblock, Muhao Chen, Binh Vu, and Jay Pujara. 2021. SPADE: A semi-supervised probabilistic approach for detecting errors in tables. In *IJCAI*. 3543–3551.
- [36] Theodoros Rekatsinas, Xu Chu, Ihab F. Ilyas, and Christopher Ré. 2017. HoloClean: Holistic data repairs with probabilistic inference. *Proceedings of the VLDB Endowment* 10, 11 (2017), 1190–1201.
- [37] El Kindi Rezig, Mourad Ouzzani, Walid G. Aref, Ahmed K. Elmagarmid, Ahmed R. Mahmood, and Michael Stonebraker. 2021. Horizon: Scalable dependency-driven data cleaning. *Proceedings of the VLDB Endowment* 14, 11 (2021), 2546–2554.
- [38] Victor Sanh, Colin Raffel Albert Webson, Stephen H. Bach, Lintang Sutawika, et al. 2022. Multitask prompted training enables zero-shot task generalization. *ArXiv Preprint ArXiv:2110.08207* (2022).
- [39] Petros Venetis, Alon Halevy, Jayant Madhavan, Marius Pasca, Warren Shen, Fei Wu, Gengxin Miao, and Chung Wu. 2011. Recovering semantics of tables on the web. *Proceedings of the VLDB Endowment* 4, 9 (2011), 528–538. <https://doi.org/10.14778/2002938.2002939>
- [40] Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V. Le. 2022. Finetuned Language Models Are Zero-Shot Learners. *ArXiv Preprint ArXiv:2109.01652* (2022).
- [41] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. 2022. Emergent abilities of large language models. *ArXiv Preprint ArXiv:2206.07682* (2022).
- [42] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2023. Chain-of-thought prompting elicits reasoning in large language models. *ArXiv Preprint ArXiv:2201.11903* (2023).
- [43] Ryan Whitcomb. 2024. Billionaires dataset. <https://corgis-edu.github.io/corgis/csv/billionaires/>. Accessed: 2024-05-18.
- [44] Yangyang Wu, Xiaoye Miao, Xinyu Huang, and Jianwei Yin. 2023. Jointly imputing multi-view data with optimal transport. In *AAAI*. 4747–4755.
- [45] Mohamed Yakout, Laure Berti-Equille, and Ahmed K. Elmagarmid. 2013. Don't be scared: Use scalable automatic repairing with maximal likelihood and bounded changes. In *SIGMOD*. 553–564.
- [46] Zihuai Zhao, Wenqi Fan, Jiatong Li, Yunqing Liu, Xiaowei Mei, Yiqi Wang, Zhen Wen, Fei Wang, Xiangyu Zhao, Jiliang Tang, and Qing Li. 2024. Recommender systems in the era of large language models (LLMs). *ArXiv Preprint ArXiv:2307.02046* (2024).